

Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación

**Nombre de la materia:** Análisis y Diseño de Algoritmos

NRC: 14443

**Titular de la materia:** Iván Olmos Pineda

**Tarea 2: Análisis práctico de algoritmos de ordenamiento (Bubble Sort,  
Insertion Sort, Quick Sort, Merge Sort, Radix Sort)**

**Alumno:**

Manuel Fernández Mercado

**Otoño 2024**

## Tabla de contenido

|                              |           |
|------------------------------|-----------|
| Marco Teórico .....          | 1         |
| Bubble Sort .....            | 1         |
| Insertion Sort .....         | 2         |
| Quick Sort .....             | 4         |
| Merge Sort .....             | 7         |
| Radix Sort .....             | 10        |
| Objetivo .....               | 13        |
| Pasos .....                  | 13        |
| Ejecución .....              | 14        |
| Análisis de resultados ..... | 16        |
| Bubble Sort .....            | 16        |
| Insertion Sort .....         | 19        |
| Merge Sort .....             | 24        |
| Quick Sort .....             | 26        |
| Radix Sort .....             | 28        |
| Conclusiones .....           | 34        |
| <b>Referencias .....</b>     | <b>35</b> |

## Marco Teórico

Para empezar, hay que hacer un pequeño análisis de complejidad a los algoritmos de ordenamiento que se van a abordar. Por términos prácticos no se explicará extensamente el funcionamiento de cada uno de los algoritmos y se usarán técnicas no exactas para conocer su costo temporal.

### Bubble Sort

El siguiente código corresponde a una versión optimizada implementada en C++ obtenida de (GeeksforGeeks, 2024).

```
void bubbleSort(vector<int>& arr) {  
    int n = arr.size();  
    bool swapped;  
    for (int i = 0; i < n - 1; i++) {  
        swapped = false;  
        for (int j = 0; j < n - i - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                swap(arr[j], arr[j + 1]);  
                swapped = true;  
            }  
        }  
        // If no two elements were swapped, then break  
        if (!swapped)  
            break;  
    }  
}
```

}

El mejor caso de este algoritmo sucede cuando el arreglo está completamente ordenado, no es necesario realizar intercambios, por lo que es un solo recorrido del arreglo el algoritmo termina. De esta forma su complejidad algorítmica es  $\Omega(n)$ .

Su peor caso sucede cuando el arreglo está en orden inverso, De esta forma en la primera pasada se hacen  $(n-1)$  comparaciones, en la segunda  $(n-2)$ , hasta llegar a 1. Obtenemos la siguiente expresión:

$$(n-1) + (n-2) + \dots + 1 = (n-1) \frac{(n-1+1)}{2} = \frac{n(n-1)}{2}$$

Por lo tanto, su orden de complejidad en el peor caso es  $O(n^2)$ .

El caso promedio ocurre cuando tiene algún porcentaje de desorden mayor que 0 y menor que 100%. Dada la naturaleza del algoritmo el número de comparaciones es constante, por lo tanto, el caso promedio se ubica en  $\theta(n^2)$ .

## Insertion Sort

Este código corresponde a la versión normal del algoritmo con un ciclo while. La implementación está realizada en C++ y fue obtenida de (GeeksforGeeks, 2024).

```
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;
```

```

        /* Move elements of arr[0..i-1], that are
           greater than key, to one position ahead
           of their current position */
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];

            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

```

El mejor caso sucede cuando el arreglo esta ordenado, de esta forma la instrucción `arr[j] > key` es falsa en cada ocasión que se ingrese al ciclo `while` por lo que solo son necesarias  $(n-1)$  recorridos para finalizar el algoritmo. Por lo tanto, la complejidad algorítmica es  $\Omega(n)$ .

El peor caso sucede cuando el arreglo esta ordenado inversamente, de esta forma cada elemento debe ser comparado y posiblemente intercambiado por cada uno de los elementos anteriores, por lo que su complejidad es  $O(n^2)$ .

El caso promedio ocurre cuando el arreglo tiene algún porcentaje de desorden. Aunque el número exacto de comparaciones para cada elemento puede variar, la complejidad sigue siendo cuadrática  $\theta(n^2)$ .

## Quick Sort

El siguiente código es la versión recursiva clásica (pivote en el último elemento del array) implementada en C++ obtenida de (GeeksforGeeks, 2024).

```
int partition(vector<int>& arr, int low, int high) {  
    // Choose the pivot  
    int pivot = arr[high];  
  
    // Index of smaller element and indicates  
    // the right position of pivot found so far  
    int i = low - 1;  
  
    // Traverse arr[low..high] and move all smaller  
    // elements on left side. Elements from low to  
    // i are smaller after every iteration  
    for (int j = low; j <= high - 1; j++) {  
        if (arr[j] < pivot) {  
            i++;  
            swap(arr[i], arr[j]);  
        }  
    }  
  
    // Move pivot after smaller elements and  
    // return its position  
    swap(arr[i + 1], arr[high]);  
    return i + 1;  
}
```

```
// The QuickSort function implementation

void quickSort(vector<int>& arr, int low, int high) {

    if (low < high) {

        // pi is the partition return index of pivot

        int pi = partition(arr, low, high);

        // Recursion calls for smaller elements

        // and greater or equals elements

        quickSort(arr, low, pi - 1);

        quickSort(arr, pi + 1, high);

    }

}
```

Dada la naturaleza *divide-and-conquer* de este algoritmo el mejor caso se presenta cuando el pivote divide al arreglo exactamente a la mitad, es decir, los árboles de recursión en la división principal son exactamente iguales. La función de recurrencia queda de la siguiente manera

$$f(n) = \begin{cases} O(1); n = 0, n = 1 \\ f\left(\frac{n}{2}\right) + O(n); n > 1 \end{cases}$$

Donde:  $f\left(\frac{n}{2}\right)$  es la parte de *divide-and-conquer* y  $O(n)$  es el costo de la función `partition`.

Aplicando el teorema maestro para recurrencias, el orden de complejidad es  $O(n * \log(n))$ .

El peor caso entonces sucede cuando el arreglo está ordenado inversamente, en esta situación el pivote dividirá cada arreglo en un subarreglo de tamaño 1 y otro de tamaño (n-1) haciendo que una sola rama de la recursión sea la más pesada, La función de recursión es la siguiente:

$$f(n) = \begin{cases} O(1); n = 0, n = 1 \\ f(n-1) + O(n); n > 1 \end{cases}$$

Aplicando el método de sustitución y generalizando la expansión después de k-pasos, tenemos la siguiente expresión:

$$f(n) = f(n-k) + O(n) + O(n-1) + O(n-2) + \dots + O(n-k+1)$$

La expansión se detiene cuando n-k=1, lo que significa que k=n-1. Sustituyendo obtenemos que:

$$f(n) = f(1) + O(n) + O(n-1) + O(n-2) + \dots + O(1)$$

Realizando la suma observamos que existe una serie aritmética y que f(1) puede ser sustituido como O(1). De esta forma y realizando algunas simplificaciones se tiene que:

$$f(n) = O\left(\frac{n(n-1)}{2}\right) + O(1) = O(n^2) + O(1) = O(n^2)$$

Por lo tanto, el peor caso es  $O(n^2)$

El caso promedio es muy parecido a el mejor caso, pero tomando en cuenta que las divisiones del arreglo pueden no ser siempre exactamente a la mitad, con esto podemos generalizar que su orden de complejidad  $\theta(n * \log(n))$



## Merge Sort

El siguiente código es la versión recursiva básica implementada en C++ obtenida de (GeeksforGeeks, 2025).

```
void merge(vector<int>& arr, int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temp vectors
    vector<int> L(n1), R(n2);

    // Copy data to temp vectors L[] and R[]
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0;
    int k = left;

    // Merge the temp vectors back
    // into arr[left..right]
    while (i < n1 && j < n2) {
```

```

        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of L[],
    // if there are any
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy the remaining elements of R[],
    // if there are any
    while (j < n2) {
        arr[k] = R[j];

```

```

        j++;

        k++;

    }

}

// begin is for left index and end is right index
// of the sub-array of arr to be sorted
void mergeSort(vector<int>& arr, int left, int right)
{
    if (left >= right)
        return;

    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

```

Este algoritmo sigue el concepto de divide-and-conquer y no es sensible a los datos, ya que siempre divide a los arreglos en dos partes sin importar su contenido, su función de recursión es la siguiente:

$$f(n) = \begin{cases} O(1); n = 0, n = 1 \\ f\left(\frac{n}{2}\right) + O(n); n > 1 \end{cases}$$

Por el teorema maestro, y al ser no sensible a los datos tenemos que el costo de la recurrencia es  $O(n * \log(n))$  para todos los casos.

## Radix Sort

El siguiente código es la versión LSD (*Less Significant Digit*) implementada en C++ obtenida de (GeeksforGeeks, 2024).

```
int getMax(int arr[], int n)
{
    int mx = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > mx)
            mx = arr[i];
    return mx;
}

// A function to do counting sort of arr[]
// according to the digit
// represented by exp.
void countSort(int arr[], int n, int exp)
{
    // Output array
    int output[n];
```

```

int i, count[10] = { 0 };

// Store count of occurrences
// in count[]
for (i = 0; i < n; i++)
    count[(arr[i] / exp) % 10]++;

// Change count[i] so that count[i]
// now contains actual position
// of this digit in output[]
for (i = 1; i < 10; i++)
    count[i] += count[i - 1];

// Build the output array
for (i = n - 1; i >= 0; i--) {
    output[count[(arr[i] / exp) % 10] - 1] = arr[i];
    count[(arr[i] / exp) % 10]--;
}

// Copy the output array to arr[],
// so that arr[] now contains sorted
// numbers according to current digit
for (i = 0; i < n; i++)

```

```

        arr[i] = output[i];
    }

// The main function to that sorts arr[]
// of size n using Radix Sort
void radixsort(int arr[], int n)
{
    // Find the maximum number to
    // know number of digits
    int m = getMax(arr, n);

    // Do counting sort for every digit.
    // Note that instead of passing digit
    // number, exp is passed. exp is 10^i
    // where i is current digit number
    for (int exp = 1; m / exp > 0; exp *= 10)
        countSort(arr, n, exp);
}

```

A diferencia de los algoritmos vistos anteriormente radix no es comparativo y dentro de el emplea otro método de ordenamiento Count Sort. Haciendo uso del método de Barómetro, vemos que Count Sort tiene costo lineal asociado a n de  $O(n+k)$

donde  $n$  es el número de datos y  $k$  es el rango de posibles valores para cada dígito (en este caso 10).

De esta forma Radix también se comporta de forma lineal pues la cantidad de veces que se ejecuta Count Sort depende la cantidad de dígitos ( $d$ ) del número más grande perteneciente al arreglo a ordenar. Por lo tanto, su costo algorítmico es  $O(d*(n+k))$ . Todos los casos se comportan de la misma manera, pero dependiendo de la naturaleza de los dígitos a ordenar se puede comportar de forma ligeramente diferente.

### **Objetivo**

El objetivo del siguiente trabajo es realizar un análisis de complejidad empírica de los métodos de ordenamiento mencionados en el marco teórico para poder contrastar la teoría con la práctica a través de una serie de experimentos.

### **Pasos**

Para poder realizar este análisis es necesario determinar una secuencia de dimensiones de arreglo lo suficientemente grandes para que el tiempo de ejecución pueda ser medido con claridad. Para garantizar un poco de certeza en el análisis necesario generar una muestra de considerable con una cierta distribución de desorden para tener acceso a distintos comportamientos de los algoritmos.

En este trabajo la muestra de arreglos será de tamaño 1000 y será distribuida de la siguiente manera: 200 con 10% de desorden, 300 con 50% de desorden y 500 con 100% de desorden.

Con el fin de que todos los algoritmos estén en igualdad de condiciones, cada muestra de arreglos será ingresada a c/u de los algoritmos mencionados, los resultados de en términos de tiempo de ejecución serán guardados en archivos de texto para su posterior análisis mediante gráficos.

## **Ejecución**

Para empezar, se diseñaron dos programas en C++, el primero de ellos genera las instancias de los arreglos en archivos de texto y el otro les pasa dichas instancias a los distintos algoritmos de ordenamiento. Los códigos vistos en el marco teórico fueron modificados para poder admitir la estructura de datos `vector`.

La longitud de los arreglos se definió mediante una serie de experimentos con Bubble Sort en su peor caso. Después de comparar algunos tiempos de ejecución de los algoritmos con las mismas instancias se decidió comenzar con un tamaño de 10,000 y avanzar de 5,000 en 5,000 hasta finalizar con un tamaño de 60,000. La cota superior fue elegida para no tener grandes tiempos de ejecución con Bubble Sort, ya que desde tamaño de arreglo el tiempo de ejecución aproximado es mayor a 3 horas; por otro lado, la cota inferior se seleccionó para que los demás algoritmos alcanzaran al menos milésimas de segundo en ejecución. Posteriormente generé los archivos de texto con los tamaños de arreglo y las 1000 instancias distribuidas en los porcentajes de desorden comentados con anterioridad.

El entorno en donde se desarrollaron los experimentos fue una computadora de escritorio con las siguientes características:

- AMD Ryzen 5500



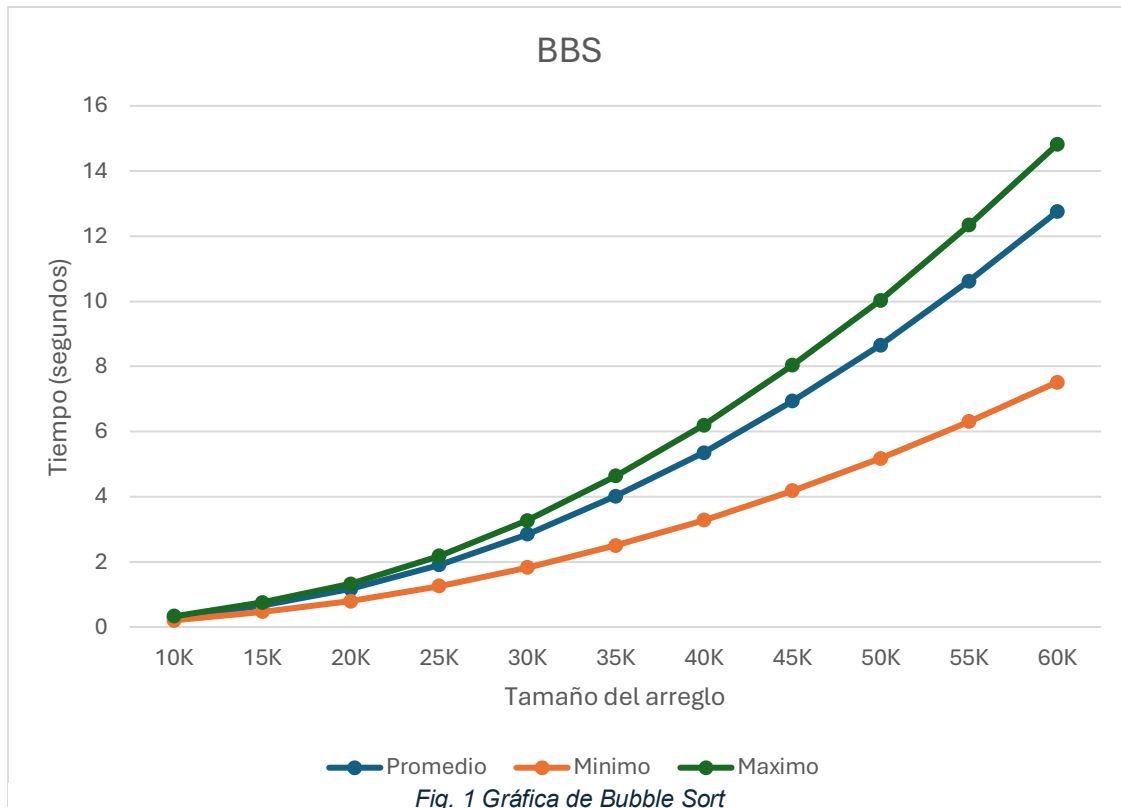
- 16 Gb RAM DDR4 3200 Mhz
- Windows 10 Pro-64 bits

Con el fin de obtener un mejor rendimiento se desactivó el Hyper Threading, se seleccionó el plan de energía Alto rendimiento y se deshabilitaron procesos en segundo plano para que el programa tuviera cierta prioridad de ejecución.

Una vez obtenidos los tiempos de ejecución de los algoritmos con los archivos de texto los exporté en hojas de Excel para poder obtener el tiempo máximo, mínimo, promedio y la varianza de cada instancia de ejecución con el fin de poder generar gráficas que permitan visualizar de una mejor manera los resultados obtenidos.

## Análisis de resultados

### Bubble Sort



De acuerdo con el marco teórico su complejidad algorítmica es la siguiente:

- Mejor caso:  $\Omega(n)$ .
- Peor caso:  $O(n^2)$ .
- Caso promedio:  $\Theta(n^2)$ .

Por la elección de porcentajes de desorden discutida en (Pasos) no fue posible alcanzar el mejor caso de bubble sort (arreglo ya ordenado), por lo que las tres funciones observadas en la gráfica se encuentran teóricamente en un orden de complejidad cuadrático.

Para comprobar que en efecto sean funciones cuadráticas se puede emplear una herramienta de Excel para asociar una línea de tendencia a cada función, dado que esperamos que sea cuadrática, se eligió que la aproximación fuera a una función polinómica de segundo grado.

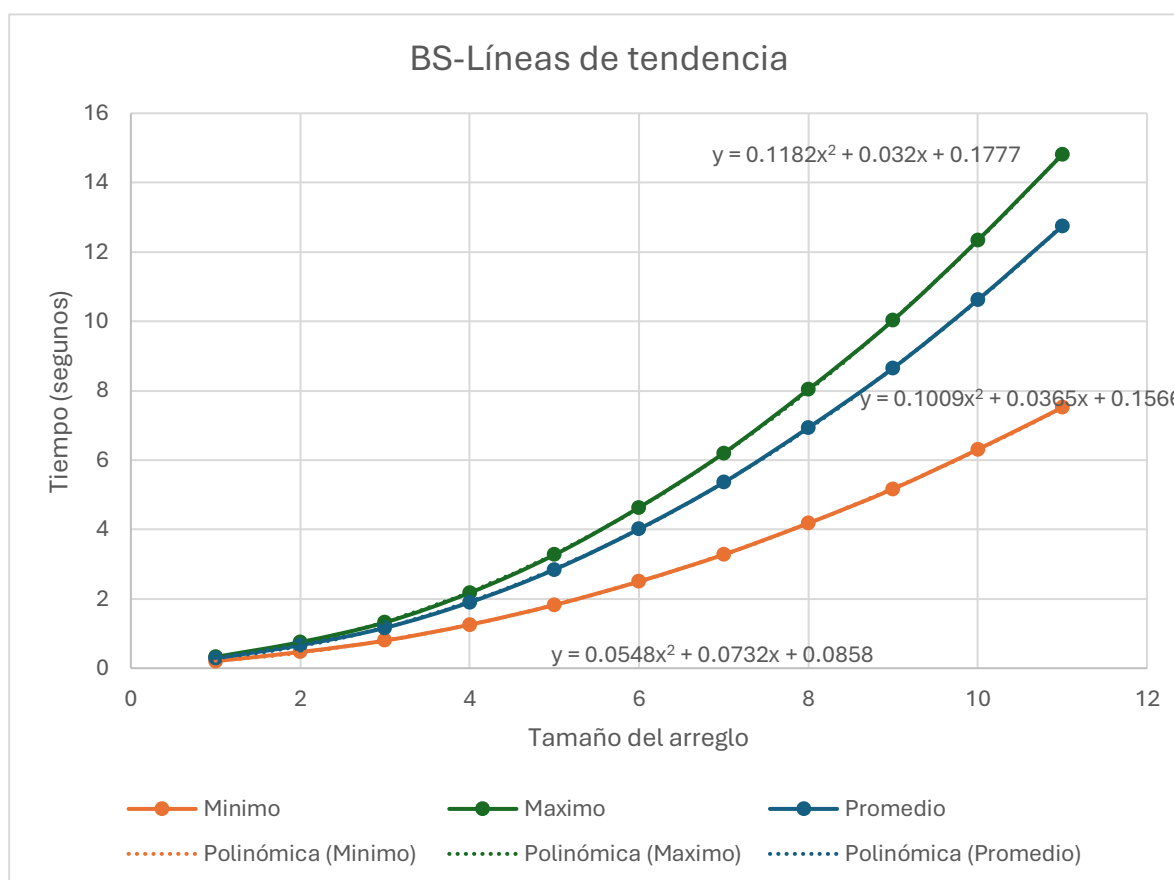
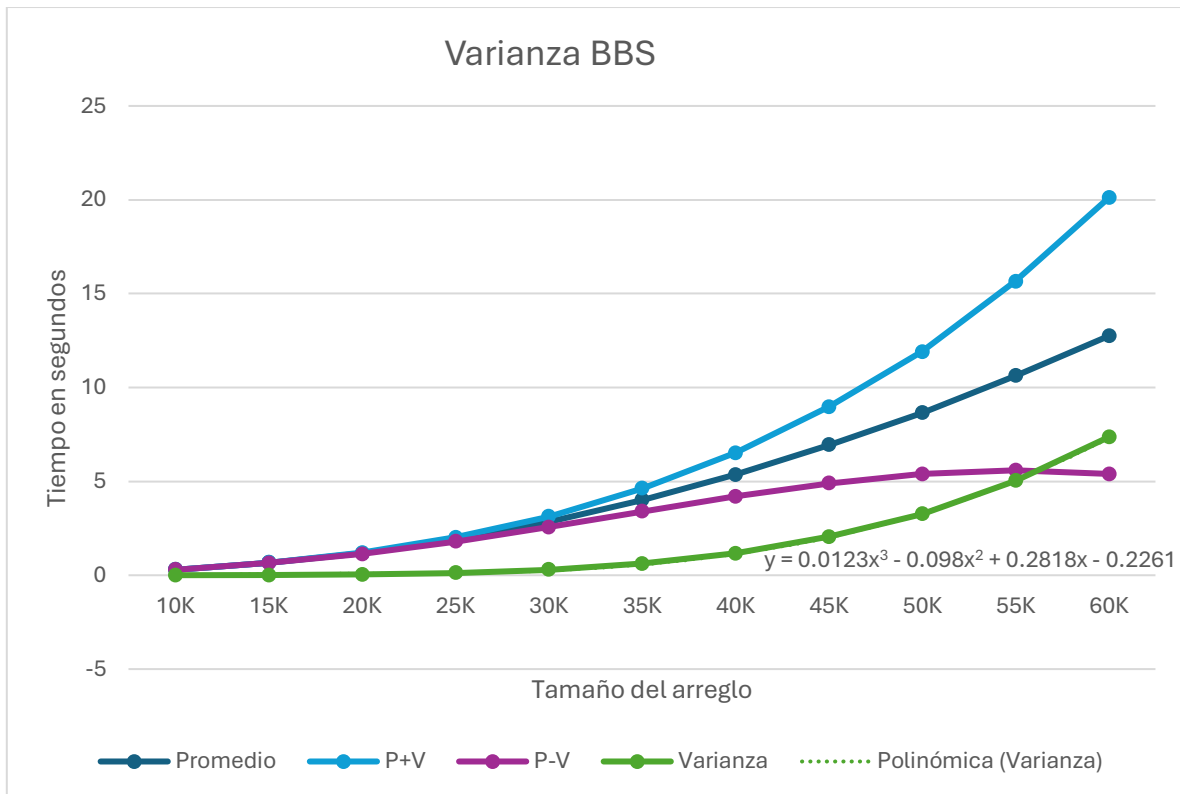


Fig. 2 Líneas de Tendencia para Bubble Sort

La aproximación que hace Excel es bastante buena ya que las líneas puntuadas son opacadas por las líneas gruesas. De esa forma comprobamos que, en efecto, la teoría coincide con la práctica.

Ahora bien, observando la Fig.1 podemos apreciar que entre menor sea el tamaño del arreglo, las funciones comienzan a estar más juntas mientras que, entre mayor

sea el tamaño de los arreglos más se separan, es decir, la varianza es proporcional en algún sentido a el tamaño del arreglo. Esto queda más claro observando el siguiente gráfico:



La distribución de los datos respecto al promedio es proporcional al tamaño del arreglo de forma cúbica, lo cual puede verse en la aproximación a la función Varianza que genera Excel. Esto nos da una idea de que Bubble Sort es más eficiente entre menor sea el número de elementos a procesar y muy ineficiente conforme el tamaño de los arreglos a procesar aumente.

## Insertion Sort

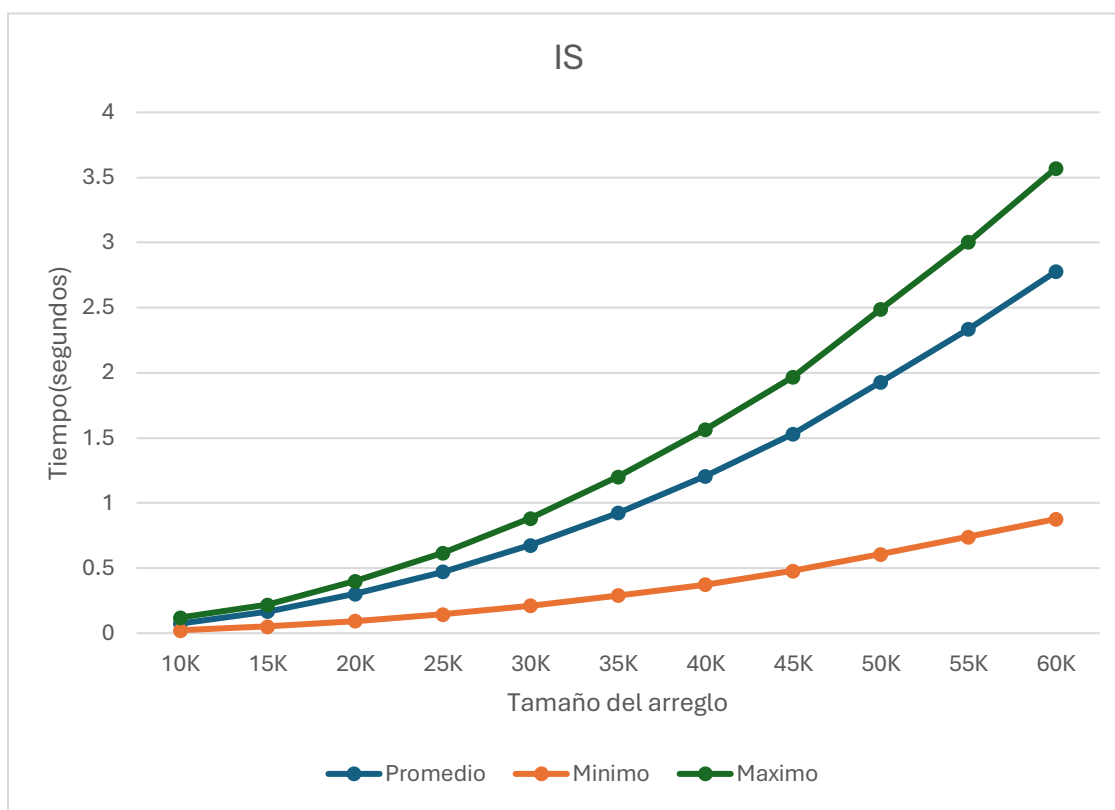


Fig. 3 Gráfica de Insertion Sort

De acuerdo con el marco teórico su complejidad algorítmica es la siguiente:

- Mejor caso:  $\Omega(n)$ .
- Peor caso:  $O(n^2)$ .
- Caso promedio:  $\Theta(n^2)$ .

Dada la elección de porcentajes de desorden discutida en (Pasos) el mejor caso de insertion Sort (arreglo ya ordenado) no fue alcanzable, por lo tanto, las tres funciones observadas en la Fig. 3 deberían corresponder a un orden cuadrático

Para comprobarlo se vuelve a hacer uso de la herramienta de Excel para asociar una línea de tendencia a cada función, dado que esperamos que las funciones sean cuadráticas se eligió una tendencia polinómica de segundo grado.

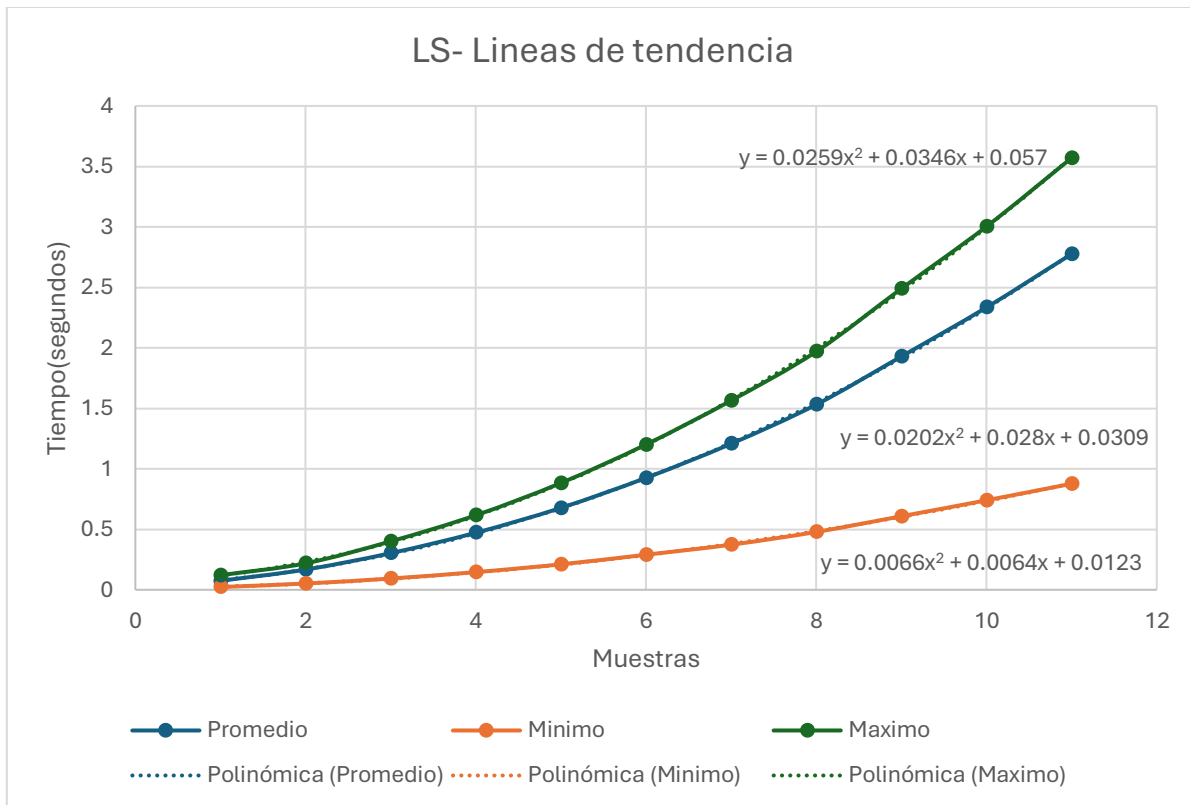


Fig. 4 Lineas de tendencia Insertion Sort

Veamos ahora hagamos un análisis de la varianza

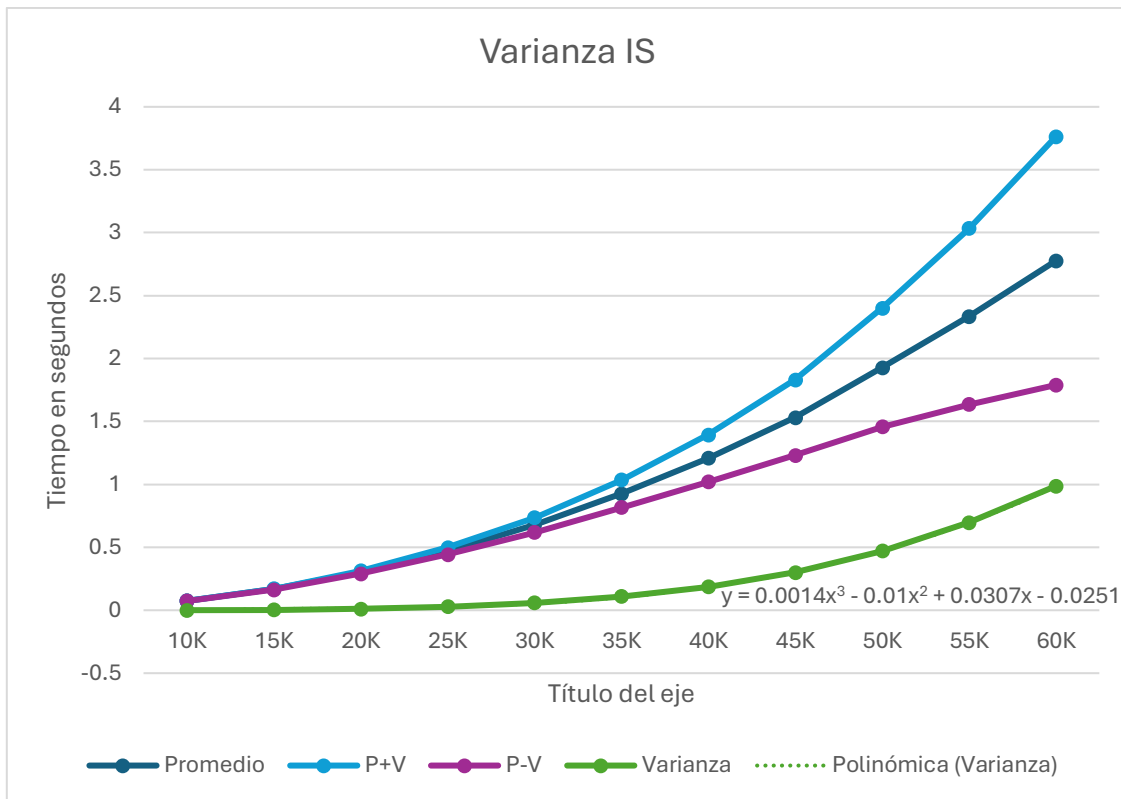


Fig. 5 Varianza Insertion Sort

Tomando en cuenta la Fig. Los datos se distribuyen más cerca del promedio mientras menor sea el tamaño del arreglo y más lejos entre mayor sea el tamaño del arreglo a procesar, es decir, el algoritmo responderá mejor entre menor sea la dimensión del arreglo y peor entre mayor sea. (igual que en Bubble Sort). Sin embargo, Insertion Sort es más eficiente que Bubble Sort por dos razones. La primera es que su varianza crece menos rápido y la segunda, que sus tiempos de ejecución son menores para todos los casos probados. Las siguientes gráficas contrastan los resultados obtenidos.

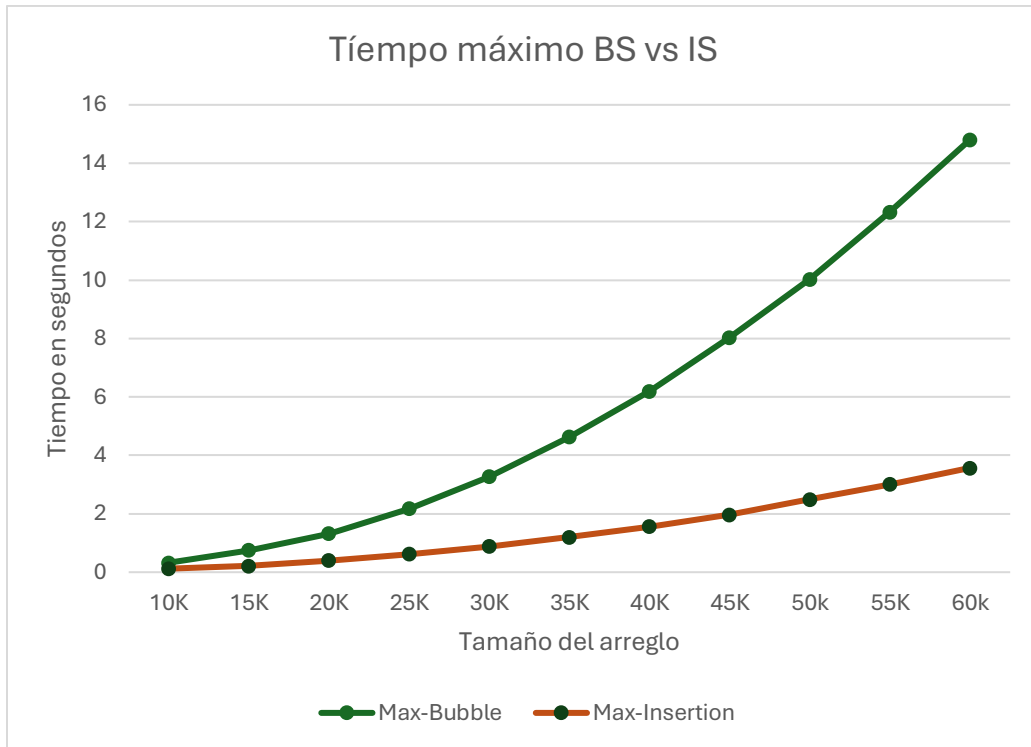


Fig. 6 Tiempo Máximo Bubble Sort vs Insertion Sort

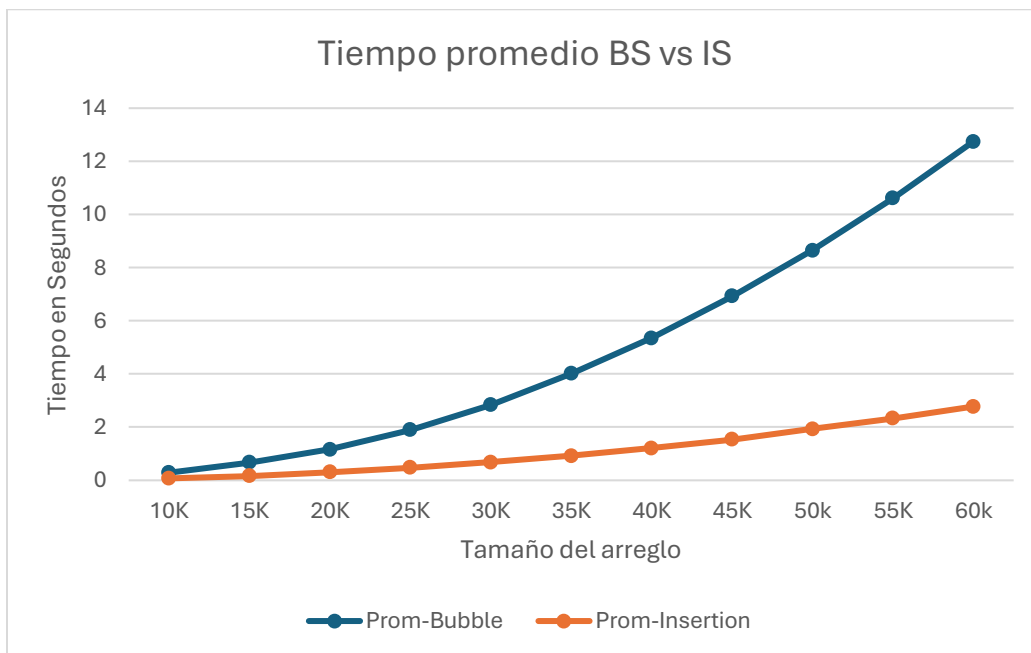


Fig. 7 Tiempo promedio Bubble Sort vs Insertion Sort



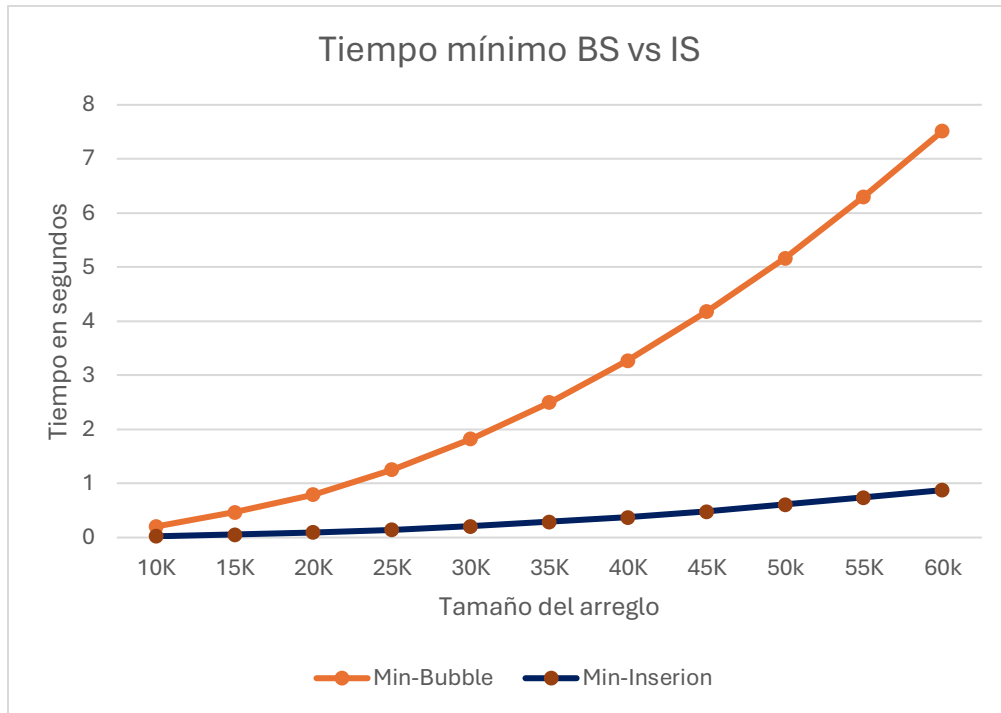


Fig. 8 Tiempo mínimo Bubble Sort vs Insertion Sort

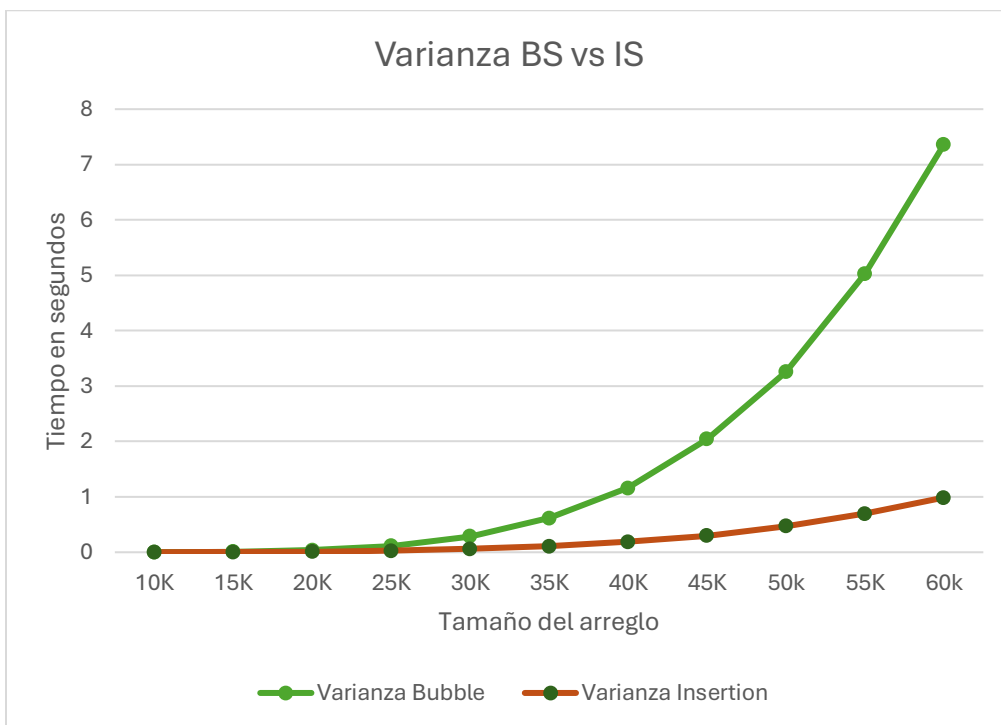


Fig. 9 Varianza Bubble Sort vs Insertion Sort

## Merge Sort

De acuerdo con el marco teórico su complejidad algorítmica es la siguiente:

- Mejor caso:  $\Omega(n * \log(n))$ .
- Peor caso:  $O(n * \log(n))$ .
- Caso promedio:  $\Theta(n * \log(n))$ .

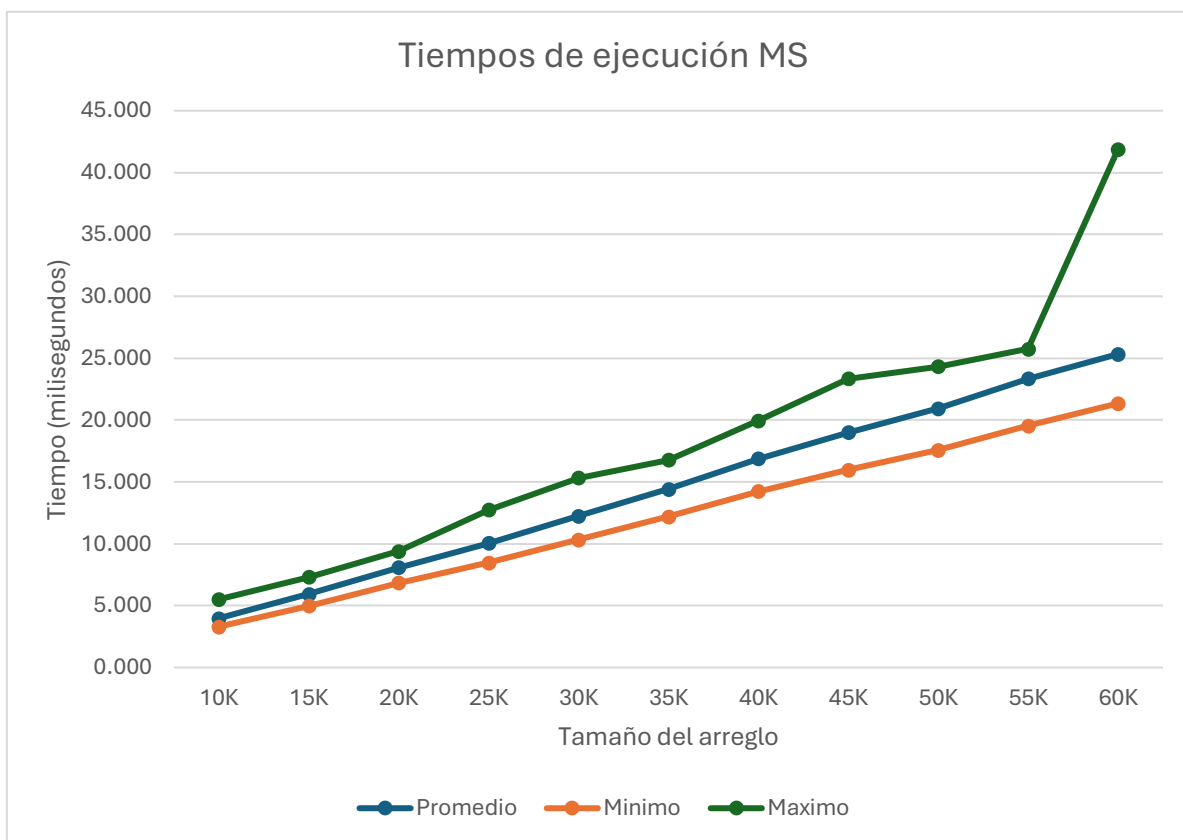


Fig. 10 Grafica de tiempos de ejecución Merge Sort

Tomando en cuenta que todos los casos tienen la misma complejidad se esperaría que todas las funciones tengan la forma de  $n * \log(n)$ . Sin embargo, no hay una forma precisa de aproximar una función con una línea de tendencia de la forma  $n * \log(n)$  en Excel. Lo que si podemos apreciar es un comportamiento lineal, el cual

tiene un crecimiento algo parecido a lo que tendríamos con  $n * \log(n)$ . Con esta información no podemos asegurar con certeza que el análisis teórico coincida con el práctico, pero se tiene cierta aproximación.

De acuerdo con la Fig.10 se observa que el peor, mejor y caso promedio no están muy lejanos entre sí. Analizando la varianza podemos percatarnos que es muy pequeña (picos segundos) para todos los casos experimentados. Esta información que nos garantiza que gran parte de los resultados se encuentren cerca del promedio y que el tiempo de ejecución sea bastante consistente para el rango de datos seleccionado.

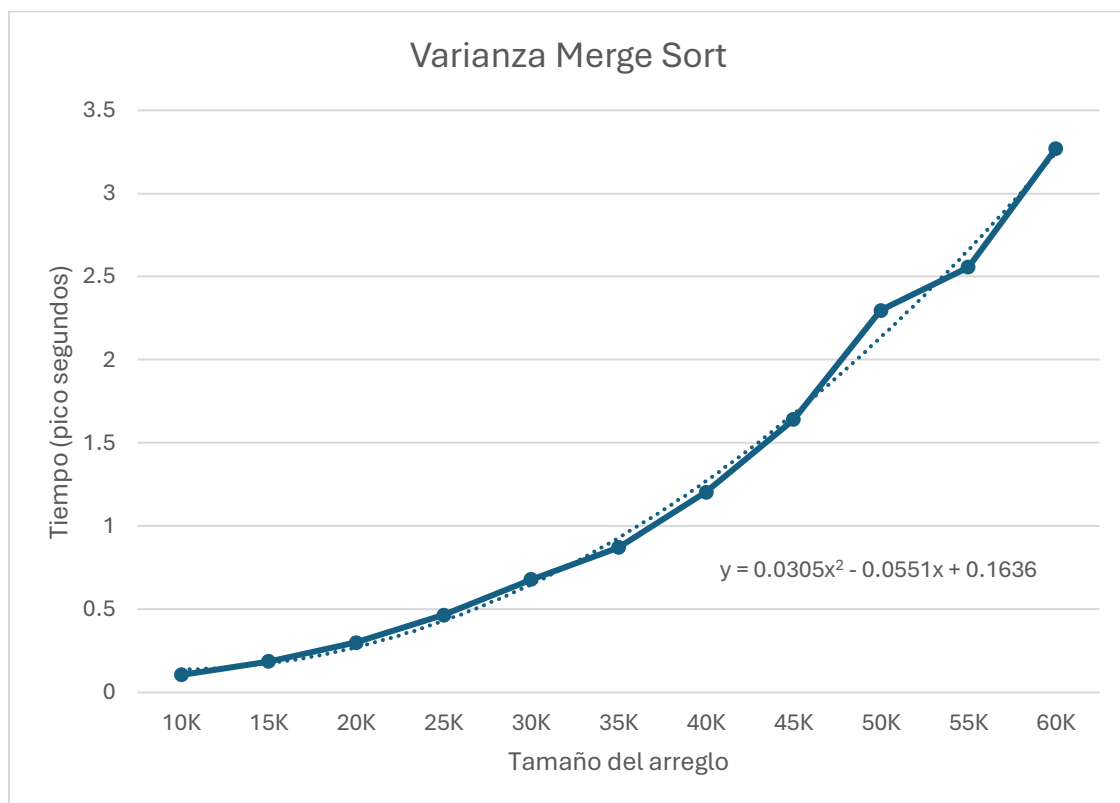


Fig. 11 Varianza de Merge Sort y línea de tendencia aproximada

Consultando las gráficas de tiempo de ejecución de los tres algoritmos vistos hasta el momento es claro apreciar que Merge Sort es mejor para el rango de datos seleccionado, sin embargo, otro dato contundente que lo comprueba es que la varianza de Merge Sort se comporta aproximadamente de forma cuadrada mientras que para Bubble Sort e Insertion Sort se comporta de forma cúbica.

## Quick Sort

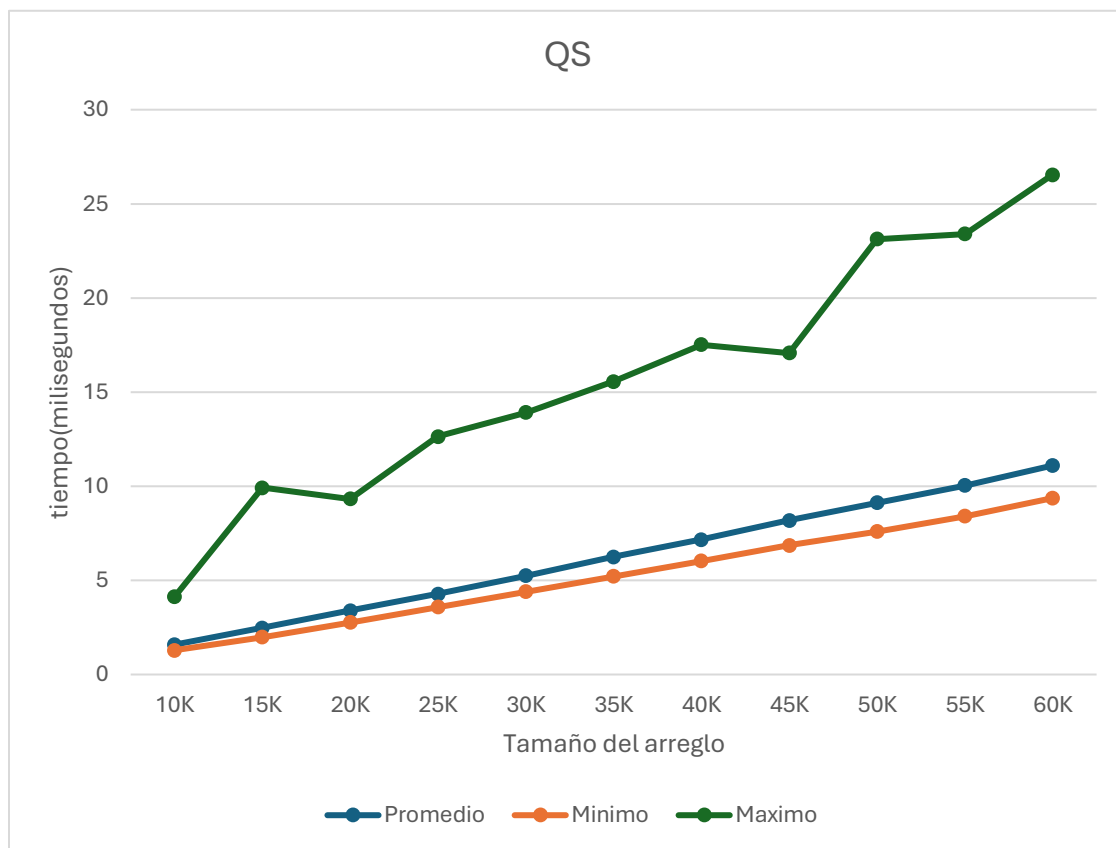


Fig. 12 Gráfica de Quick Sort

De acuerdo con el marco teórico su costo algorítmico es:

- Mejor caso:  $\Omega(n * \log(n))$ .

- Peor caso:  $O(n^2)$ .
- Caso promedio:  $\Theta(n * \log(n))$ .

Debido a la naturaleza de los porcentajes de desorden no fue posible alcanzar el peor caso del algoritmo. De esta forma se esperaba que el mejor que las funciones de la gráfica se comportaran como  $n * \log(n)$ . Sin embargo, como se mencionó en Merge Sort, no existe una forma de verificar al 100% que las funciones generadas sean realmente  $n * \log(n)$ , por lo que la aproximación lineal es lo mejor que se puede tener.

Si bien QuickSort es más rápido que Merge Sort en los casos promedio, Quick Sort es menos consistente. Esto puede comprobarse contrastando que la función máxima en Quick Sort (Fig.12) está más lejana al promedio que la máxima en Merge Sort. Parte de esta idea también puede ratificarse comparando los valores de las varianzas para cada algoritmo, siendo la de Merge Sort menor a la de Quick Sort.

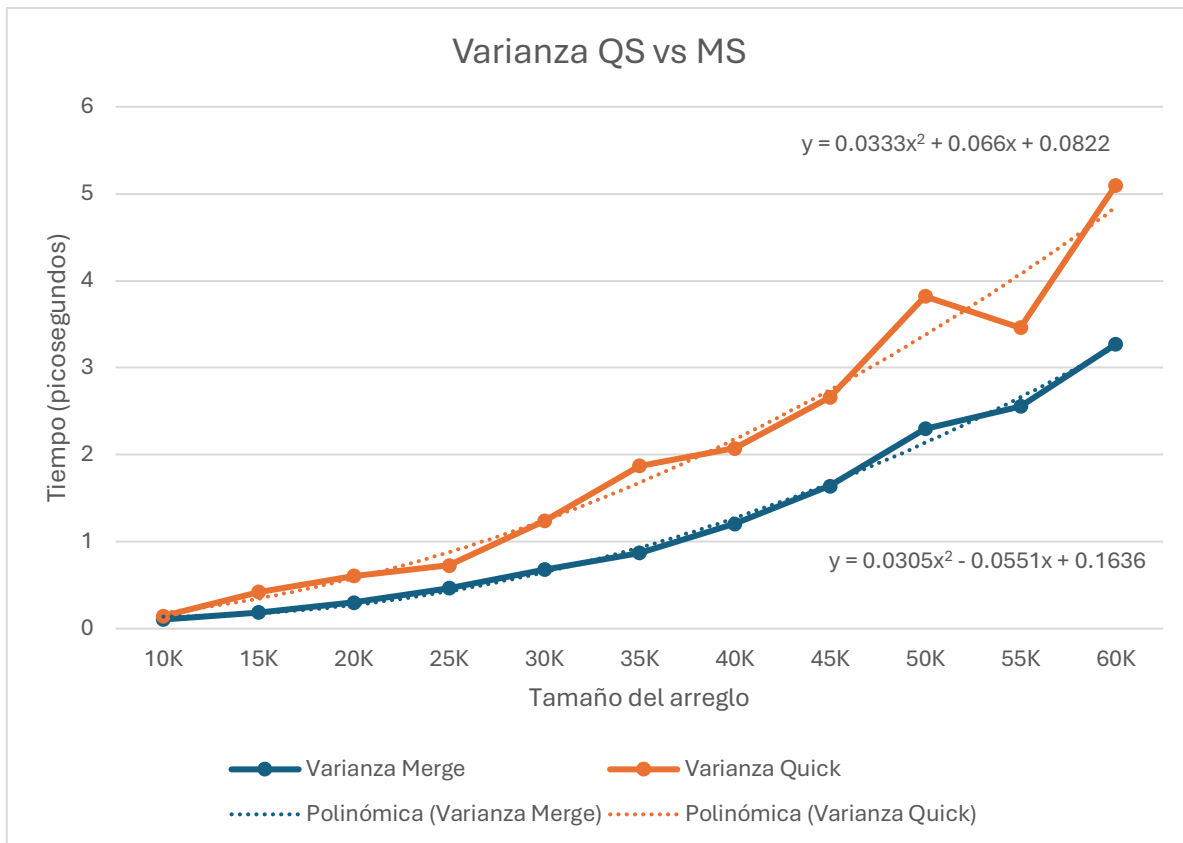


Fig. 13 Varianza de Quick Sort y Merge Sort con las líneas de tendencia relacionadas

## Radix Sort

De acuerdo con la teoría su complejidad algorítmica es la siguiente:

- Mejor caso:  $\Omega(dn)$ .
- Peor caso:  $O(dn)$ .
- Caso promedio:  $\Theta(dn)$ .

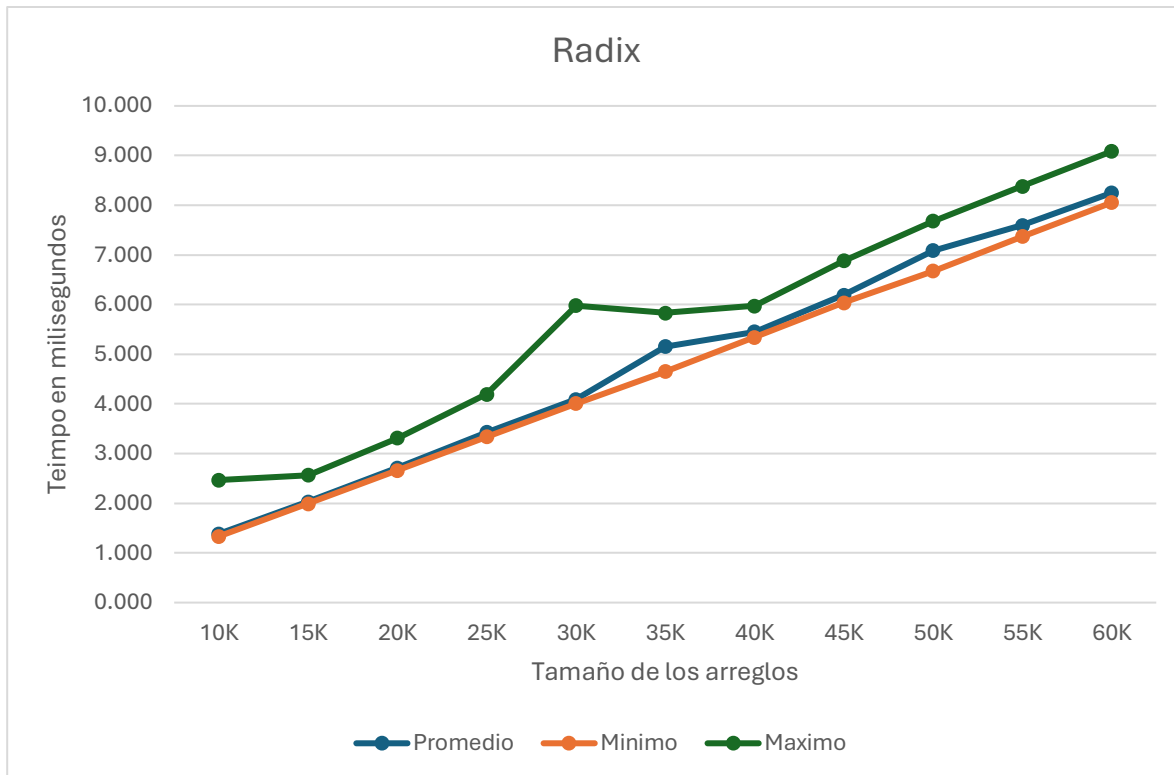


Fig. 14 Gráfica de tiempo de ejecución Radix Sort

Dado que este algoritmo es lineal respecto al número de datos a procesar, podemos esperar que en la experimentación realizada encontremos los tres casos. Para comprobar que las funciones observadas en la Fig.14 sean lineales se asociará una línea de tendencia lineal a c/u con ayuda de Excel.

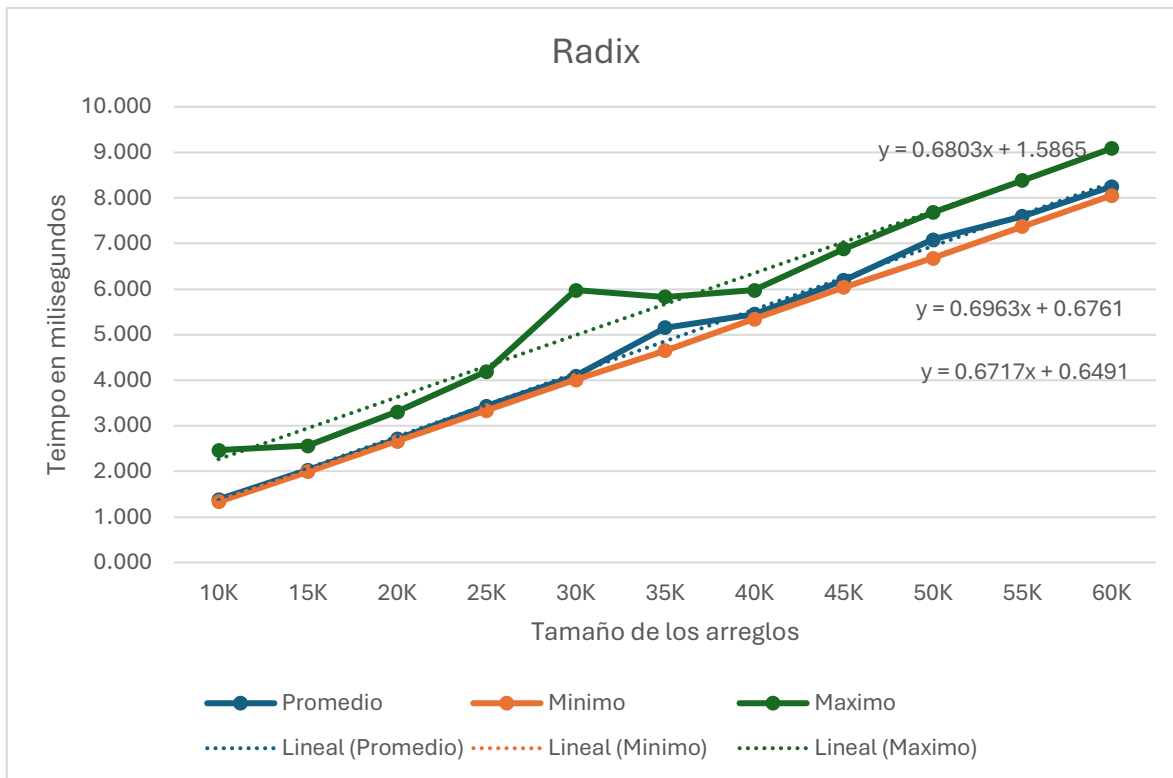
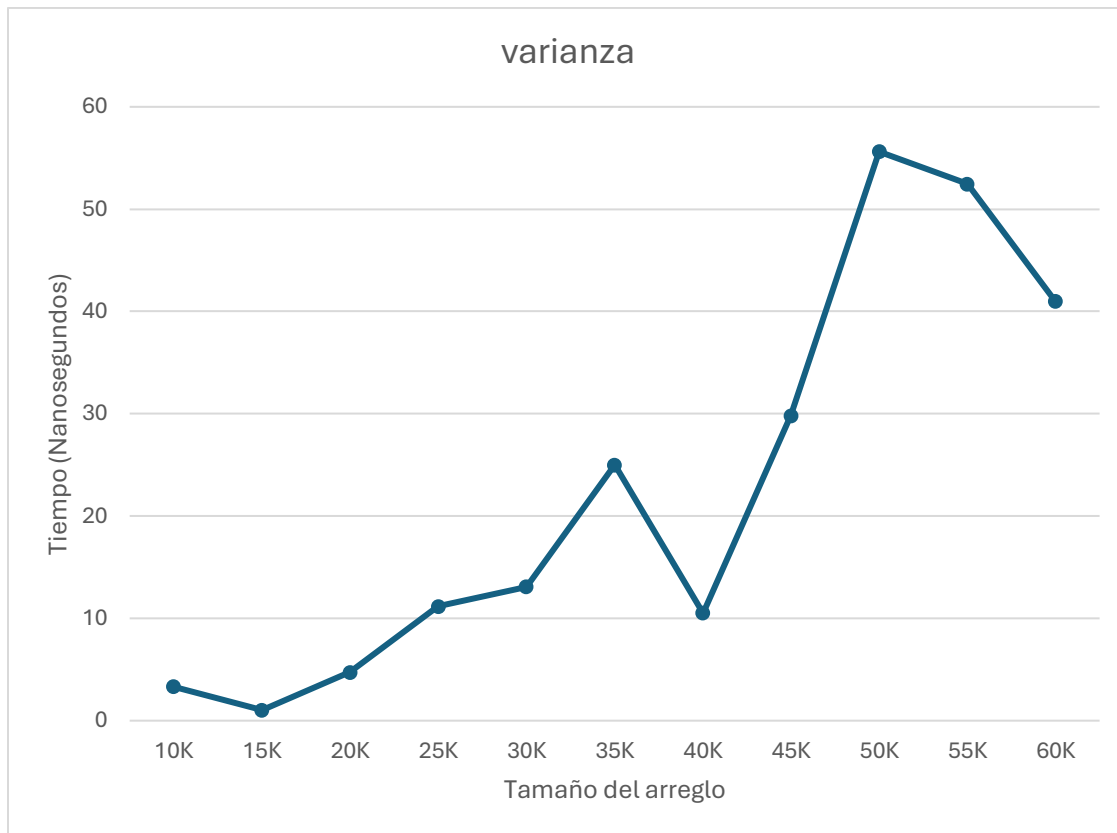


Fig. 15 Lineas de tendencia Radix (ordenadas de arriba a abajo)

En esencia cada función se comporta de forma lineal con algunas ligeras variaciones. De esta forma podemos decir que la práctica coincide con la teoría.

Ahora bien, tomando la misma Fig. 14 podemos ver que todas las funciones están bastante próximas a las demás (sobre todo promedio y mínimo), lo cual es un buen referente para su rendimiento, de la misma manera, las líneas de tendencia nos hacen esperar que el algoritmo se comporte igual de bien para datos más grandes o pequeños. Por otra parte, las varianzas son muy pequeñas (nanosegundos) para todos los casos probados, pero se comportan de forma extraña. Quizás esto se deba a ámbitos espaciales del algoritmo.





*Fig. 16 Varianza de Radix Sort*

esto se puede comprobar calculando las varianzas (véase anexo 5) las cuales son bastante pequeñas para todos los casos. Esta información nos indica que el algoritmo es bastante consistente por lo que podría comportarse igual de bien para cantidades de datos más grandes y pequeñas.

En comparación con todos los algoritmos vistos con anterioridad Radix es el más eficiente con respecto al tiempo de ejecución, a continuación, se mostrarán algunas gráficas que cotejan estos resultados (se omiten las comparaciones con Bubble Sort e Insertion Sort por la escala, al igual que las comparaciones con la varianza).

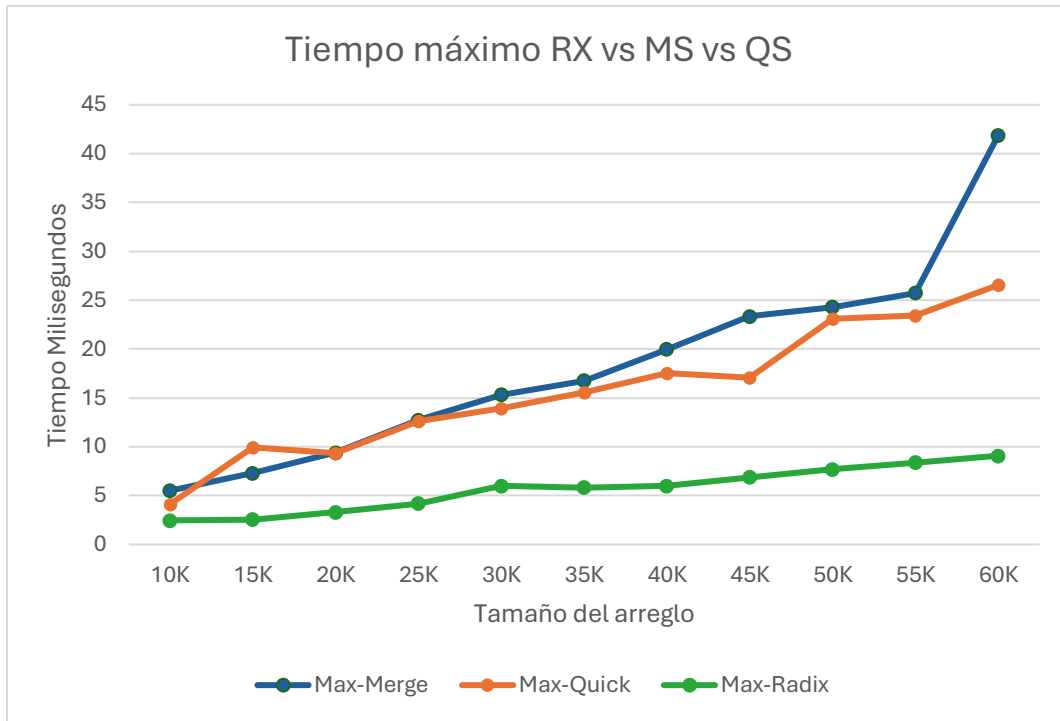


Fig. 17 Tiempo Máximo Radix Sort vs Merge Sort vs Quick Sort

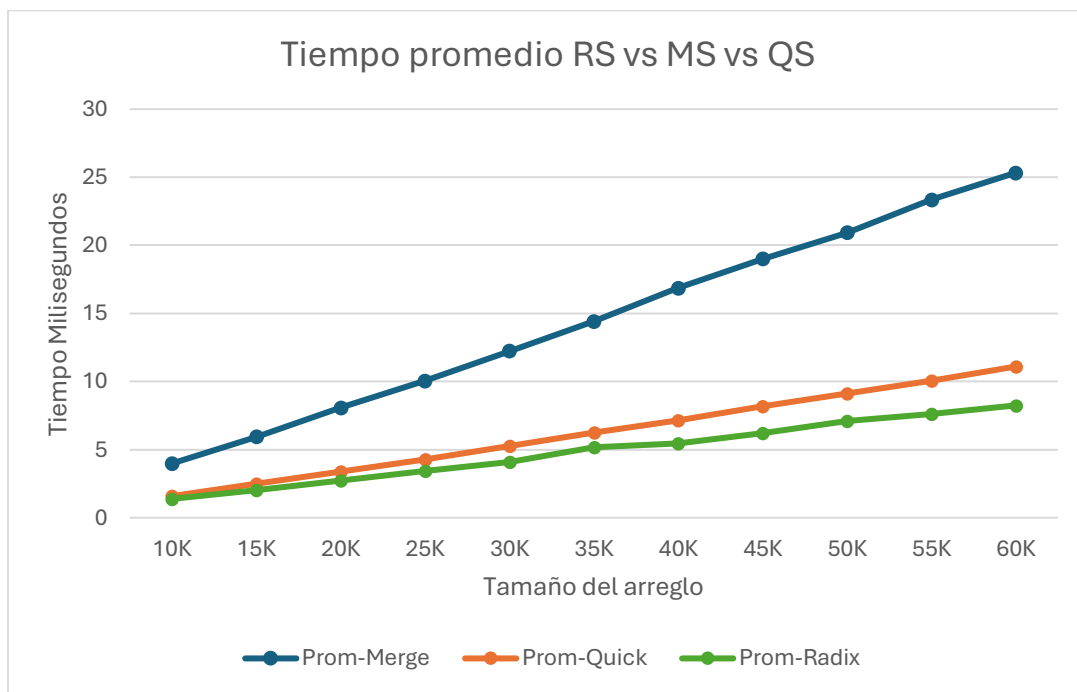
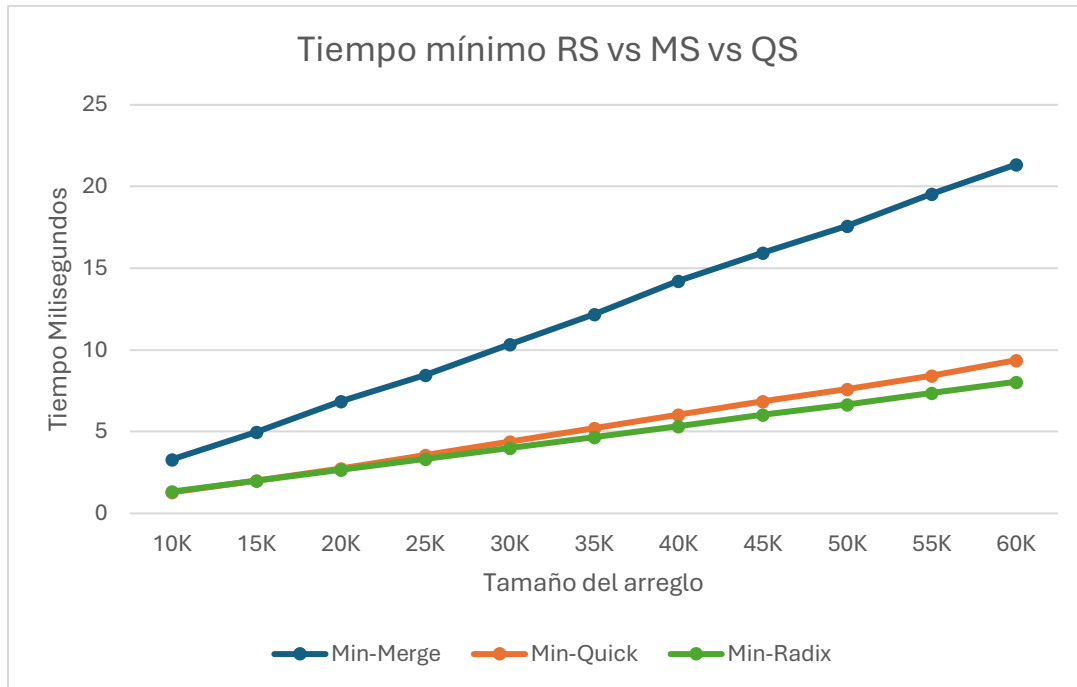


Fig. 18 Tiempo Promedio Radix Sort vs Merge Sort vs Quick Sort



*Fig. 19 Tiempo Minimo Radix Sort vs Merge Sort vs Quick Sort*

En este conjunto de gráficas apreciamos que los comportamientos de Merge Sort y Radix Sort son bastante consistentes a través del máximo, mínimo y promedio (no cambian mucho) de todos los rangos de arreglos. Por otro lado, Quick Sort puede ser una buena alternativa al estar bastante cerca de Radix Sort, sin embargo, sus resultados en el caso máximo pueden no ser los mejores; quizás un control sobre el porcentaje de desorden sea la clave para que Quick Sort tenga un buen desempeño.

## Conclusiones

La teoría se contrastó muy bien con la práctica para el caso de Bubble Sort e insertion Sort debido a la elección del tamaño y cantidad de arreglos, gracias a ello se pudo comprobar de buena manera su comportamiento general. Sin embargo Quick Sort y Merge Sort no vieron tan beneficiados, ya que no fue posible observar gráficamente el comportamiento logarítmico, esto debido probablemente a los bajos tiempos de ejecución registrados en la experimentos así como a la cantidad de muestras elegidas, aún así el análisis posterior mostró algunas curiosidades de cada uno. Algo parecido sucedió con Radix Sort , si bien el comportamiento teórico coincidió con la práctica, los bajos tiempos obtenidos así como la increíblemente pequeña escala de la varianza complicó realizar un análisis un poco más detallado como en los otros algoritmos. contrastó bien la teoría con la práctica pero su comportamiento deja.

La experimentación dejó en claro que los algoritmos con complejidad cuadrática sin embargo, para Quick Sort y Merge Sort no fueron suficientes para observar gráficamente el comportamiento de su mejor, peor y caso promedio. Para Merge, Quick y Radix sería necesario hacer más pruebas con conjuntos de datos más grandes.

La experimentación y contrastación de resultados fue de gran ayuda para conocer el comportamiento general de los algoritmos vistos lo cual es gran punto de referencia para su elección en situaciones específicas. Este trabajo puede ser un punto de partida para un análisis más detallado y extenso de los algoritmos

Se pudo apreciar que en a la mayoría de los casos la parte teórica coincidió con la práctica, sin embargo, no dadas las condiciones temporales no fue posible, sería necesario empezar a analizar arreglos extremadamente

## Referencias

GeeksforGeeks. (6 de Octubre de 2024). *Bubble Sort Algorithm*. Obtenido de GeeksforGeeks: <https://www.geeksforgeeks.org/bubble-sort-algorithm/>

GeeksforGeeks. (7 de Octubre de 2024). *Insertion Sort Algorithm*. Obtenido de GeeksforGeeks: <https://www.geeksforgeeks.org/insertion-sort-algorithm/>

GeeksforGeeks. (3 de Octubre de 2024). *Quick Sort*. Obtenido de GeeksforGeeks: <https://www.geeksforgeeks.org/quick-sort-algorithm/>

GeeksforGeeks. (29 de Agosto de 2024). *Radix Sort – Data Structures and Algorithms Tutorials*. Obtenido de GeeksforGeeks: <https://www.geeksforgeeks.org/radix-sort/>

GeeksforGeeks. (18 de Marzo de 2024). *Time and Space Complexity Analysis of Bubble Sort*. Obtenido de GeeksforGeeks: <https://www.geeksforgeeks.org/time-and-space-complexity-analysis-of-bubble-sort/>

GeeksforGeeks. (14 de Marzo de 2024). *Time and Space Complexity Analysis of Quick Sort*. Obtenido de GeeksforGeeks: <https://www.geeksforgeeks.org/time-and-space-complexity-analysis-of-quick-sort/>

GeeksforGeeks. (8 de Febrero de 2024). *Time and Space Complexity of Insertion Sort*. Obtenido de GeeksforGeeks: <https://www.geeksforgeeks.org/time-and-space-complexity-of-insertion-sort-algorithm/>

GeeksforGeeks. (2024). *Time and Space complexity of Radix Sort Algorithm*. Obtenido de GeeksforGeeks: <https://www.geeksforgeeks.org/time-and-space-complexity-of-radix-sort-algorithm/>

GeeksforGeeks. (17 de Septiembre de 2025). *Merge Sort – Data Structure and Algorithms Tutorials*. Obtenido de GeeksforGeeks: <https://www.geeksforgeeks.org/merge-sort/>

Anexos

## Anexos

Las siguientes tablas están redondeadas a tres decimales, para consultar las tablas completas al igual que las gráficas es necesario consultar el archivo de Excel anexado al trabajo.

Anexo 1: Tabla para Bubble Sort (segundos)

| Tamaño   | 10K   | 15K   | 20K   | 25K   | 30K   | 35K   | 40K   | 45K   | 50K    | 55K    | 60K    |
|----------|-------|-------|-------|-------|-------|-------|-------|-------|--------|--------|--------|
| Promedio | 0.288 | 0.664 | 1.163 | 1.897 | 2.842 | 4.011 | 5.353 | 6.934 | 8.650  | 10.617 | 12.747 |
| Minimo   | 0.204 | 0.466 | 0.795 | 1.254 | 1.821 | 2.499 | 3.276 | 4.180 | 5.168  | 6.305  | 7.516  |
| Maximo   | 0.325 | 0.751 | 1.320 | 2.172 | 3.268 | 4.632 | 6.199 | 8.037 | 10.032 | 12.335 | 14.811 |

Anexo 2: Tabla para Insertion Sort (segundos)

| Tamaño   | 10K   | 15K   | 20K   | 25K   | 30K   | 35K   | 40K   | 45K   | 50K   | 55K   | 60K   |
|----------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| Promedio | 0.074 | 0.167 | 0.303 | 0.472 | 0.677 | 0.925 | 1.207 | 1.531 | 1.929 | 2.335 | 2.777 |
| Minimo   | 0.023 | 0.052 | 0.093 | 0.146 | 0.210 | 0.290 | 0.373 | 0.478 | 0.608 | 0.740 | 0.877 |
| Maximo   | 0.121 | 0.219 | 0.400 | 0.616 | 0.883 | 1.201 | 1.565 | 1.970 | 2.490 | 3.003 | 3.571 |

Anexo 3: Tabla para Merge Sort (milisegundos)

| Tamaño   | 10K   | 15K   | 20K   | 25K    | 30K    | 35K    | 40K    | 45K    | 50K    | 55K    | 60K    |
|----------|-------|-------|-------|--------|--------|--------|--------|--------|--------|--------|--------|
| Promedio | 3.981 | 5.946 | 8.078 | 10.058 | 12.244 | 14.418 | 16.868 | 19.003 | 20.931 | 23.337 | 25.330 |
| Minimo   | 3.305 | 4.977 | 6.841 | 8.458  | 10.329 | 12.194 | 14.234 | 15.959 | 17.575 | 19.537 | 21.344 |
| Maximo   | 5.509 | 7.305 | 9.399 | 12.729 | 15.312 | 16.764 | 19.963 | 23.344 | 24.315 | 25.751 | 41.873 |

Anexo 4: Tabla para Quick Sort (milisegundos)

| Tamaño   | 10K   | 15K   | 20K   | 25K    | 30K    | 35K    | 40K    | 45K    | 50K    | 55K    | 60K    |
|----------|-------|-------|-------|--------|--------|--------|--------|--------|--------|--------|--------|
| Promedio | 1.587 | 2.475 | 3.391 | 4.276  | 5.249  | 6.254  | 7.169  | 8.195  | 9.118  | 10.041 | 11.102 |
| Minimo   | 1.290 | 1.988 | 2.764 | 3.585  | 4.391  | 5.220  | 6.027  | 6.858  | 7.589  | 8.417  | 9.371  |
| Maximo   | 4.132 | 9.923 | 9.323 | 12.640 | 13.922 | 15.557 | 17.513 | 17.075 | 23.130 | 23.407 | 26.552 |

Anexo 5: Tabla para Radix Sort (milisegundos)

| Tamaño   | 10K   | 15K   | 20K   | 25K   | 30K   | 35K   | 40K   | 45K   | 50K   | 55K   | 60K   |
|----------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| Promedio | 1.384 | 2.030 | 2.714 | 3.432 | 4.094 | 5.157 | 5.451 | 6.193 | 7.086 | 7.599 | 8.248 |
| Minimo   | 1.333 | 1.994 | 2.661 | 3.339 | 4.009 | 4.650 | 5.345 | 6.039 | 6.677 | 7.371 | 8.053 |
| Maximo   | 2.467 | 2.565 | 3.311 | 4.195 | 5.979 | 5.830 | 5.976 | 6.885 | 7.681 | 8.382 | 9.084 |



De acuerdo con el marco teórico, sabemos que el